Map internals

What's actually happening under `m := map[K]V{}`. The hash table structure, the bucket layout, the growth and evacuation algorithm, and the iteration randomization mechanism.

The hmap header

A map variable is a single pointer. The actual hash table lives behind it.

```
// runtime/map.go (simplified)
type hmap struct {
    count     int            // number of live entries (== len(m))
    flags     uint8
    B         uint8          // log2 of bucket count; total buckets = 1 << B
    noverflow uint16         // approximate overflow bucket count
    hash0     uint32         // hash seed (randomized per-map)

    buckets    unsafe.Pointer // pointer to []bmap; size = 2^B
    oldbuckets unsafe.Pointer // non-nil only during growth
    nevacuate  uintptr        // progress through evacuation

    extra *mapextra           // overflow bucket bookkeeping
}
```

`make(map[K]V)` allocates an `hmap` and an initial bucket array. `m == nil` is true only when the variable itself is nil; an empty map has a non-nil pointer.

Buckets

Each bucket holds up to 8 key/value pairs. The bucket struct is generated by the compiler per `(K, V)` pair, so the layout is type-specific.

```
// conceptual layout per bucket
type bmap struct {
    tophash  [8]uint8     // top 8 bits of hash, or special marker
    keys     [8]K
    values   [8]V
    overflow *bmap        // next bucket in this chain, or nil
}
```

Why 8 slots: chosen empirically as a sweet spot between cache-line use and probing cost. A bucket plus its `tophash` array fits in roughly one or two cache lines for small `K` and `V` types.

tophash markers

The `tophash` byte for each slot tells the runtime how to interpret that slot:

Value	Meaning
0	empty (never used)
1	empty after delete; slot reusable
2-4	special evacuation states
5+	normal: top 8 bits of the key's hash

When looking up, the runtime first scans the `tophash` array for matches. The actual key comparison only happens for slots with a matching top-byte. This makes the inner loop very cache-friendly.

Hashing

Every map gets a per-map random seed (`hash0`) at creation time. This randomizes the hash function so an attacker can't construct collision-heavy key sets.

```
hash := alg.hash(key, uintptr(m.hash0))
```

The hash algorithm depends on architecture and key type: - On x86/amd64, `aeshash` uses hardware AES instructions for speed. - On platforms without AES, `memhash` (a custom hash) is used.

The hash output is split: - Top 8 bits go into `tophash` for the bucket-internal fast filter. - The low `B` bits pick the bucket: `bucket = hash & ((1 << B) - 1)` .

The bucket count is always a power of 2 (`2^B`). This is why the low-bit selection is a cheap mask, not a modulo.

Lookup algorithm

For `v, ok := m[key]` :

1. Compute `hash = h(key, m.hash0)` .
2. Pick the bucket: `b = m.buckets[hash & mask]` .
3. Compute the top byte: `top = hash >> (sizeof(uintptr)*8 - 8)` , clamped to be at least 5.
4. Scan the bucket's `tophash[]` . For each match:
    - Compare the stored key with the lookup key.
    - On match, return the value and `ok=true` .

5. If no match in this bucket, follow the `overflow` pointer to the next bucket and repeat.
6. If we hit a nil overflow, return the zero value and `ok=false`.

During growth, the algorithm also checks `oldbuckets` if the new bucket hasn't been evacuated yet (see Growth).

Insert

For `m[key] = value`:

1. Compute hash, bucket, top byte as above.
2. Walk the bucket chain looking for either:
   - An existing slot with this key (update path).
   - The first empty slot (insert path).
3. If found existing key: overwrite the value, return.
4. If found empty slot: write key, value, tophash; bump `m.count`.
5. If the bucket and all its overflows are full: allocate a new overflow bucket and append it.

After insertion, if the load factor or overflow count is too high, growth is triggered (see below).

Load factor and growth triggers

The map grows when either condition holds:

1. **Load factor too high.** `count > loadFactor * 2^B`. The constant is 6.5, so on average each bucket holds about 6.5 of its 8 slots before doubling.
2. **Too many overflow buckets.** Excessive chaining indicates pathological clustering even if the average density is low. The runtime triggers same-size growth in this case to re-bucket and clear overflows.

Two growth modes:

- **Double growth:** `2^B` becomes `2^(B+1)`. Used when the load factor triggers.
- **Same-size growth:** Reuses the same `B` but allocates a fresh bucket array. Used when overflow buckets pile up; the goal is to compact, not expand.

Incremental evacuation

Growth doesn't happen all at once. That would create a multi-millisecond pause for a large map. Instead, evacuation is amortized across subsequent operations.

When growth starts: 1. The current `buckets` becomes `oldbuckets`. 2. A new bucket array of the new size is allocated as `buckets`. 3. No data moves yet.

Then on every insert, delete, and (sometimes) lookup: 1. The runtime evacuates at least one old bucket as a side effect. 2. Evacuated buckets are marked in `tophash` so they're skipped on future ops.

For a double growth, each old bucket evacuates to one of two new buckets (high-bit decides). For same-size growth, it goes to the same index.

Once every old bucket is evacuated, `oldbuckets` is cleared and growth completes.

This is why map operations have an *amortized* O(1) cost. A single operation might do extra evacuation work, but the total work is bounded.

Implications

- Map operations during growth are slightly slower (have to check `oldbuckets`).
- After growth completes, the old bucket memory is garbage; GC reclaims it.
- A long-lived map that grows and then shrinks (lots of deletes) does not give memory back to the OS. The bucket array stays allocated. For shrink-heavy patterns, create a fresh map periodically.

Iteration

`for k, v := range m` walks the bucket array. The walk:

1. Picks a randomized starting bucket and a randomized starting offset within that bucket.
2. Visits every bucket in order (wrapping around at the end).
3. Visits all 8 slots per bucket, then follows overflows.
4. Skips empty slots and evacuated-to-elsewhere slots.

The randomization makes iteration order unpredictable across runs and across maps. This is intentional. The runtime randomizes even for a single iteration to discourage code that accidentally depends on order.

Iteration is safe with concurrent reads, but **not** with concurrent writes. The runtime detects concurrent writes during iteration (in race-detection mode and sometimes outside) and crashes the program. Don't write to a map being iterated.

You *can* modify the map you're iterating, with caveats: - Deletes are visible if the key hasn't been visited yet. - Inserts may or may not be visited, depending on bucket placement. - Don't rely on either behavior.

Why you can’t take the address of a map element

```
m := map[string]int{}
&m["a"]              // compile error: cannot take address of map element
```

The reason: any insert can trigger a growth, which moves entries to new buckets. A pointer into the old buckets would dangle. The compiler prevents this entirely.

The workaround: pull the value out, modify, write back. Or use `map[K]*V` and store pointers (the pointers are heap-allocated and stable).

```
v := m["a"]; v++; m["a"] = v
```

Similarly:

```
m["a"].field = x     // compile error if value is a struct
```

The map value is a copy (a temporary). You can’t assign to a field of a temporary. Same workaround.

Why concurrent map access is unsafe

The runtime maintains internal state during operations: tophash transitions, bucket growth progress, overflow chains. Without locking, two writers can corrupt this state.

Specific failure modes: - Two concurrent inserts can leave the bucket chain in an inconsistent state. - A read during a write can see torn keys or values for non-word-sized types. - A read during evacuation can follow a stale pointer.

The runtime detects some of these and panics:

```
fatal error: concurrent map writes
fatal error: concurrent map read and map write
```

Detection isn’t perfect; it’s best-effort. Don’t rely on it.

Solutions: - Plain `map[K]V` + `sync.RWMutex`: simplest, fast in the common case. - `sync.Map`: tuned for two specific patterns (write-once-read-many, disjoint key sets per goroutine). Slower than mutex+map in general use. - Shard by hashing the key into one of N maps, each with its own mutex (reduces contention).

Memory layout numbers

Per bucket, on 64-bit, for `map[string]int`:

- tophash: 8 bytes (one byte per slot)
- keys: 8 * 16 = 128 bytes (each string is 16 bytes: ptr + len)
- values: 8 * 8 = 64 bytes (each int is 8 bytes)
- overflow: 8 bytes

Total: 208 bytes per bucket, plus alignment padding. Plus the heap memory the strings point at.

For sparse maps (load factor 1-2 per bucket), most of this is empty slots. The 8-slot-per-bucket choice is a memory tradeoff.

Compiler optimization for small maps

The Go compiler has a small-map optimization: maps with a single bucket and few entries can sometimes be represented inline without separate heap allocation. The threshold and behavior change across versions; check the compiler source if you care about a specific release.

Lookup cost in practice

Scenario	Cost
Found in first bucket, first matching tophash	~5-10 ns
Found after 1-2 tophash mismatches in same bucket	~10-20 ns
Found after 1 overflow hop	~20-30 ns
Miss (full bucket scan, no overflow)	~10-15 ns
Miss (with overflow chain)	scales with chain length

Numbers are very rough and depend on key type, hash cost, and CPU. They're dominated by memory access patterns: hot buckets stay in L1; cold buckets cost L3 or memory.

Iteration cost

Range iteration is O(n + b), where n is the entry count and b is the bucket count. Empty buckets still cost a bit because the loop visits the tophash array.

For very sparse maps (e.g., capacity hint was way too high), iteration can be surprisingly slow relative to entry count. Right-size the capacity hint when you know.

Map capacity hint

```
m := make(map[K]V, 100)
```

The hint tells the runtime to pre-size `B` so that the initial buckets can hold roughly 100 entries without growth. The actual `2^B` chosen is the smallest power of 2 where `loadFactor * 2^B >= 100`.

Benefits: - Avoids growth churn (which involves rehashing and evacuation). - Lower allocation count over the map's lifetime.

Cost: - Slightly more memory upfront for sparse usage.

Hint is approximate. The runtime may grow beyond the hint if you add more, and won't shrink below it.

Delete

`delete(m, key)` finds the slot, marks the tophash as "empty after delete," and clears the key and value (so the GC can reclaim referenced memory). It does not shrink the map. The empty slot is reusable for future inserts in the same bucket.

A pattern of "fill and clear" doesn't reclaim bucket memory. If memory matters, replace the map:

```
m = make(map[K]V, expectedSize)
```

Generic maps vs reflect

Go's map operations are statically typed. The compiler generates per-type bucket layouts at compile time, which is why iterating `map[string]int` is fast and direct.

`reflect.Map` lookups go through a slower indirect path (reflect.MapKeys, reflect.Value.MapIndex). Each operation involves type-checking and allocation. Use reflect only for genuinely dynamic code, not for "convenience."

Map vs sync.Map tradeoff

`sync.Map` has different internal structure: two maps (read-mostly + dirty) and an atomic pointer. It's tuned for cases where: - Each key is written once and read many times. - Different goroutines work on disjoint key sets.

For balanced read/write or full overlap, `sync.RWMutex` + plain map is faster, often by 2-3x. Benchmark in your access pattern; don't assume `sync.Map` is "the concurrent map."

Best practices

1. **Provide capacity hint** when the eventual size is known. `make(map[K]V, n)`.
2. **Never write to a map being iterated** by another goroutine. The race detector won't always save you.
3. **For shrink-heavy patterns, replace the map** instead of deleting in place. Memory won't return otherwise.
4. **Use `map[K]*V` if you need to mutate values in place.** The pointer is stable; the map value isn't.
5. **Pick map keys that are cheap to hash.** Long strings and large structs make every operation slower.
6. **Shard for high-contention concurrent access.** A single mutex map is a serializability bottleneck.
7. **Don't rely on iteration order, even for one iteration.** The runtime randomizes.
8. **Check `len(m) == 0`** instead of `m == nil` when you mean "empty." A make'd map is non-nil but empty.

Common gotchas

Gotcha	Cause	Fix
`nil` map write panics	Map variable never initialized	`make(map[K]V)` first
`&m["k"]` fails to compile	Map elements aren't addressable	Pull into var, or use `map[K]*V`
Struct field via map fails	Same reason	Same fix
Iteration order varies	Intentional randomization	Sort keys if you need order
Concurrent writes panic	No internal locking	RWMutex, sync.Map, or shard
Memory not released after deletes	Map doesn't shrink	Replace map
Slow iteration on tiny entries	Sparse over-hinted map	Right-size the hint
Map of slices, "shared" updates	Slice header copied on read	Read out, modify, write back
Random crash with concurrent read+write	Detected sometimes, not always	Always lock writes
Same-size growth surprising	Overflow trigger, not load factor	Check insertion pattern for clustering

---

Quick reference

Question	Answer
Map header type	`*hmap` (one pointer)
Default bucket slot count	8
Bucket count constraint	Always a power of 2 (`2^B`)
Bucket selection	`hash & (2^B - 1)`
Fast filter per slot	`tophash` (top 8 hash bits)
Hash on amd64	`aeshash` (hardware AES)
Hash fallback	`memhash`
Per-map seed	randomized `hash0`
Load factor trigger	6.5 entries per bucket average
Growth mode 1	double (`2^(B+1)`)
Growth mode 2	same-size (compact overflows)
Evacuation	incremental, amortized across ops
Iteration order	randomized intentionally
Concurrent write detection	best-effort, panics on detection
Address of element	not allowed (compile error)
Capacity hint	`make(map[K]V, n)`
Shrink on delete	no; create new map if needed